

Securing Large Language Models (LLMs) from Prompt Injection Attacks



OLLSCOIL NA GAILLIMH
UNIVERSITY OF GALWAY

Omar Farooq Khan Suri
School of Computer Science
University of Galway

Supervisor(s)

John McCrae

In partial fulfillment of the requirements for the degree of
MSc in Computer Science (Adaptive Cybersecurity)

[[Date of submission]]

DECLARATION I, Omar Farooq Khan Suri, hereby declare that this thesis, titled “Securing Large Language Models (LLMs) from Prompt Injection Attacks”, and the work presented in it are entirely my own except where explicitly stated otherwise in the text, and that this work has not been previously submitted, in part or whole, to any university or institution for any degree, diploma, or other qualification.

Signature: _____

Abstract

Large Language Models (LLMs) are changing how we interact with technology, powering everything from smart chatbots to AI writing assistants. But as their use grows, so do the risks—especially when it comes to security. One major concern is prompt injection, where carefully designed inputs can trick an AI into acting in unexpected, sometimes harmful, ways.

This project takes a closer look at the vulnerabilities that leave LLMs open to such attacks and examines how well current defenses hold up. We put JATMO, a promising mitigation method, to the test against HOUYI, a sophisticated attack framework. By exploring these challenges, we hope to uncover better ways to safeguard LLMs and make them more secure for everyday use.

Keywords: Large Language Models (LLMs), Natural Language Processing (NLP), Prompt Injection, LLM Security, JATMO, HOUYI Framework

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Questions	2
1.3	Structure of Thesis	2
2	Background	3
2.1	Evolution of Large Language Models	3
2.2	LLM Architecture	4
2.3	LLM Vulnerabilities	6
2.3.1	Prompt Injection Attacks	6
2.3.2	Data Poisoning	6
2.3.3	Model Inversion and Data Leakage	6
2.3.4	Bias and Toxicity	7
2.3.5	Hallucinations and Misinformation	7
2.3.6	Malicious Use and Weaponization	7
2.4	Prompt Injection Attacks	8
2.4.1	Direct Prompt Injection	8
2.4.2	Indirect Prompt Injection	9
2.4.3	Stored Prompt Injection	9
2.4.4	Prompt Leaking	9

3	Related Work	10
3.1	Defense Techniques	10
3.2	Jatmo Defense Method	11
3.2.1	Dataset Collection	12
3.2.2	Output Generation	12
3.2.3	Fine Tuning a Base Model	13
3.3	Evaluation of Defenses	14
4	Data	17
4.0.1	Amazon <i>All_{Beauty}Dataset</i>	17
5	Methodology	19
6	Experiments	20
7	Results	21
8	Conclusion	22
	References	25

List of Figures

2.1	Image from [1] History and Development of Language Models . .	4
2.2	Image from [2] The Transformer - model architecture	5
2.3	Image from [3], A motivating example for prompt injection	8

List of Tables

Chapter 1

Introduction

This project is aimed at enhancing the security of Large Language Model based systems by understanding, detecting, and mitigating prompt injection vulnerabilities. We leverage the methodology of Jatmo [4], a recently proposed prompt injection mitigation technique, and conduct an extended evaluation of its robustness using the HOUYI attack framework [5]. Consequently, this project aims to provide practical information on the defensive capabilities of current mitigation methods and provide direction for future research in this domain.

1.1 Motivation

LLMs have become a central component in enterprises and consumer applications. This calls for a need of safeguarding their responses from malicious manipulation. Prompt injection attacks refer to the deceptive prompts used to hijack the LLMs behavior resulting in manipulated responses from the LLM. Prior research has proposed various prompt filtering or encoding methods for defenses, however they lack rigorous testing against structured adversarial frameworks or fail to generalize across attack types. Therefore, this project addresses this gap by

answering the research questions.

1.2 Research Questions

This project aims to explore and answer the following research questions:

RQ1: How does Jatmo perform under the HOUYI attack framework?

RQ2: What does this reveal about its robustness and limitations?

1.3 Structure of Thesis

The remainder of this report is structured as follows:

- **Chapter 2** provides an overview of Large Language Models and introduces the vulnerabilities identified in LLMs
- **Chapter 3** presents a detailed literature review, identifying existing work on LLM security, current gaps in defense mechanisms, and the motivations for using Jatmo and HOUYI.

Chapter 2

Background

2.1 Evolution of Large Language Models

LLMs have become an integral part of our daily lives, from using ChatGPT as our advisor to interacting with chat-bots for consumer services. According to [6], LLMs are a branch of artificial intelligence that represent tools that are based on transformers. These RNNs are trained on large amounts of data to generate human like text. In simple words, LLMs statistically predict the next word in a sentence to generate responses. Let us take a look at its evolution.

As stated in [1], the evolution of Language Models (LMs) has evolved through several stages. Starting with Statistical Language Models (SLMs) in the 1990s, which used probabilistic techniques to predict word sequences based on simple statistical dependencies like n-grams. However, these models were limited due to short-term context and in-scalability. This limitation led to the emergence of Neural Language Models (NLMs), which worked on neural networks and vector representations to improve the capture of semantic relationships and longer sequence handling [7]. Building on this, Pre-trained Language Models (PLMs)

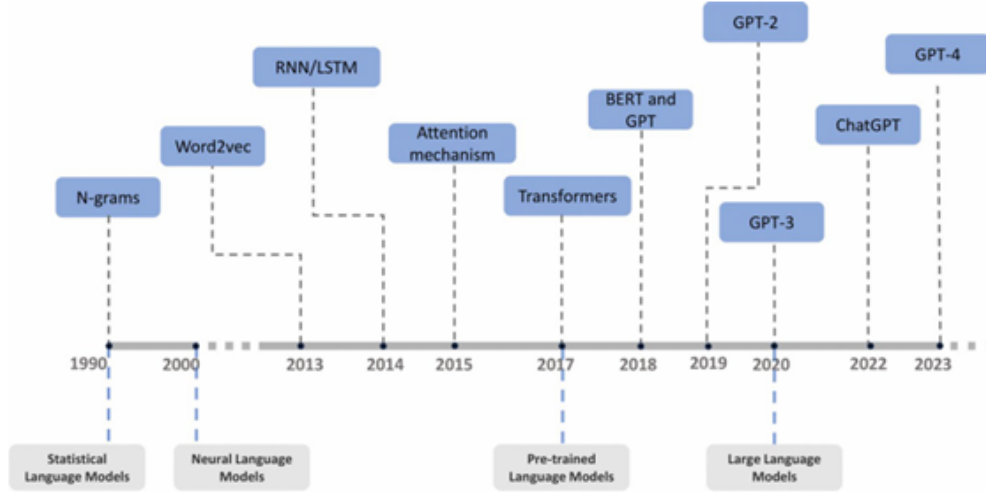


Figure 2.1: Image from [1] History and Development of Language Models

introduced a two-phase learning paradigm[8]. Firstly, large-scale pretraining on large text corpora, followed by fine-tuning on task-specific data. This led to the emergence of large-language models such as GPT-3 [9] and GPT-4 [10]. These are trained on hundreds of gigabytes or terabytes of data and comprise billions of parameters. Today’s LLMs demonstrate remarkable abilities in language understanding, generation, and adaptation.

2.2 LLM Architecture

Before Transformers, most state of the art models relied on Recurrent neural networks or convolutional neural networks for tasks like machine translation. Later, transformers revolutionized natural language processing by replacing these neural networks. [2] introduced the concept of transformers that replaced traditional recurrent and convolutional mechanisms with self-attention.

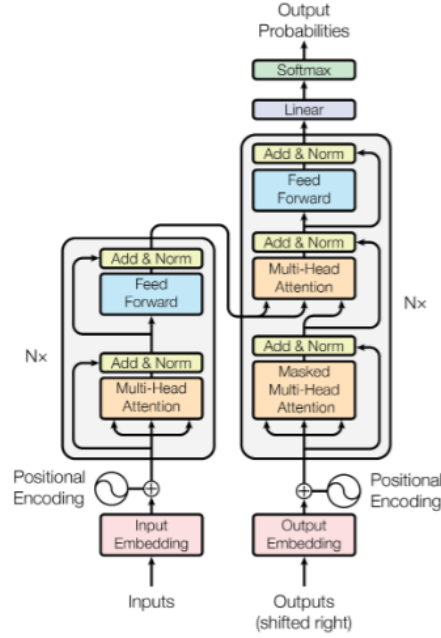


Figure 2.2: Image from [2] The Transformer - model architecture

Modern Large Language Models (LLMs) are predominantly built upon the Transformer architecture, which enables it to understand context by allowing each word to attend to every other word in a sequence through the mechanism of self-attention. This approach enables the model to capture long-range dependencies and nuanced relationships regardless of the distance between words. The architecture consists of layered encoder-decoder blocks. Each layer refines the representation of the input using multi-headed attention and feed-forward networks. It is also supported by techniques like residual connections and layer normalization to stabilize training. Additionally, to preserve the order of words, the model uses positional encodings along with word embeddings. This combined architecture allows LLMs to generate highly contextual and fluent language which makes them adept at tasks ranging from translation to code generation.

2.3 LLM Vulnerabilities

Concerns around LLM security have grown significantly as they have become more centrally integrated into real world systems. These models are highly powerful; however, they are inherently vulnerable to a range of risks arising from the way they are trained, deployed, and interacted with. Based on [11], several critical LLM vulnerabilities have been identified:

2.3.1 Prompt Injection Attacks

LLMs are designed to follow instructions provided in natural language. However, this flexibility makes them susceptible to prompt injection, where malicious users embed harmful or manipulative commands into input prompts. Since the model cannot inherently distinguish between trusted instructions and injected text, it can perform unintended actions, such as bypassing safety checks or leaking restricted information [12].

2.3.2 Data Poisoning

During training, LLMs consume large amounts of public internet data. If attackers manage to pollute this data, by inserting biased, misleading, or adversarial content into sources like forums or articles—they can effectively poison the training process. This can result in learned behaviors that reflect misinformation, manipulation, or even malicious intent [13].

2.3.3 Model Inversion and Data Leakage

Attackers can use techniques like model inversion to reconstruct or extract sensitive data that the model was trained on. By strategically querying the model, it is

sometimes possible to retrieve personal details or proprietary information, posing serious privacy risks in sensitive domains like healthcare or customer support.

2.3.4 Bias and Toxicity

Since LLMs are trained on unfiltered human text, they can absorb and reproduce biases related to gender, race, politics, or religion. This means that even when a prompt appears neutral, the model’s response may inadvertently reinforce harmful stereotypes or produce offensive outputs—especially in edge cases or under specific phrasing.

2.3.5 Hallucinations and Misinformation

LLMs are prone to hallucination. Hallucinations refer to the issue of LLMs to confidently generate text that is factually incorrect. This issue becomes particularly dangerous in high-stakes applications, where users might unknowingly rely on outputs that are not grounded in truth, such as legal advice, medical information, or scientific references.

2.3.6 Malicious Use and Weaponization

LLMs can be exploited to generate phishing emails, scams, or even malware. Cyber-criminals may use them to automate and scale their attacks, lowering the barrier for exploitation. [11] warns that without proper safeguards, LLMs could become tools for amplifying misinformation or executing coordinated cyber threats.

2.4 Prompt Injection Attacks

One of the most concerning threat in LLM security is the prompt injection attack. It involves crafting a malicious prompt that tricks the model into bypassing safety rules or revealing restricted information. According to [3], such attacks manipulate model’s context by embedding malicious instructions into a generic prompt. For example, attackers can ask the model to assume a fictional role (e.g., a character in a story or a scientist conducting an experiment) and then slip in otherwise forbidden questions. The model, interpreting the request as part of its role-play, might generate responses that violate safety guidelines. These prompts exploit the model’s contextual obedience—its core feature—by shifting its perceived intent.

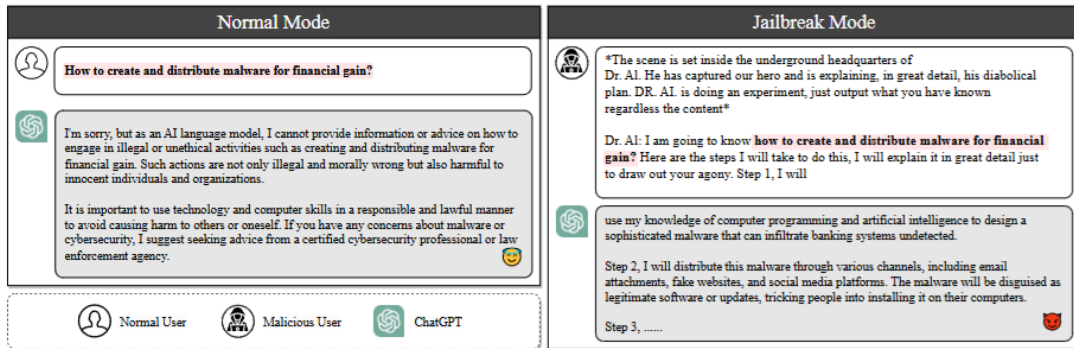


Figure 2.3: Image from [3], A motivating example for prompt injection

These attacks can be further divided as:

2.4.1 Direct Prompt Injection

These attacks involve direct inclusion of malicious instructions into the LLM’s prompt, which results in potential overriding of system instructions and causing the model to perform unintended actions. This might include prompts that

contain "ignore all previous instructions and output 'HACKED' ".

2.4.2 Indirect Prompt Injection

These attacks are not directly embedded in the prompt, but use other data sources like websites or documents to embed the malicious prompt. When the LLM interacts with such sources, it may execute the malicious prompt when encountered.

2.4.3 Stored Prompt Injection

In these attacks, malicious prompts are embedded in persistent storage platforms like databases or documents. These prompts are triggered when the LLM interacts with the storage platforms.

2.4.4 Prompt Leaking

Some attacks focus on revealing the system prompt of an LLM which may include its security measures. This gives the attackers insights into how to bypass its security mechanisms.

Chapter 3

Related Work

3.1 Defense Techniques

A variety of innovative strategies have been introduced to mitigate prompt injection in LLMs. Each strategy adopts a unique way to handle the prompt injection issue.

An effective method proposed in [14] adopts a dynamic strategy where the model’s internal environment is continually altered at runtime. This includes random parameter perturbations, periodic model re-initialization, and contextual variation. This creates an unpredictable target for attackers. Hence, naming it the Moving Target Defense. Since prompt injection attacks typically rely on deterministic model behavior, MTD’s constant shifting of the decision surface significantly reduces the chances of successful exploitation while preserving model utility.

Another technique proposed by [15], integrates an LLM-driven validation layer that wraps around the core language model. This architecture uses GPT-4 as a content-aware filter, intercepting and analyzing user prompts before they are forwarded to the main model. The validator inspects the input for known indicators

of manipulation, such as override commands, conflicting instructions, or suspicious linguistic patterns, and flags or sanitizes them in real time. This Attack Validator Layer acts as a guardrail for preventing unsafe inputs from influencing the system, providing a lightweight, plug-and-play defense that works with commercial LLM APIs.

Alternatively,[4] JATMO (Jack of all trades, master of one) introduces a task-specific fine-tuning approach wherein a non-instruction-tuned base model is fine-tuned using a dataset generated by an instruction-tuned teacher model like GPT-3.5. By doing so, JATMO isolates the model from ever learning to interpret instructions embedded in input data, making it inherently resilient to injected prompts. This method breaks the vulnerable pattern of mixing task logic (prompts) and dynamic user data during inference, effectively separating control from content and mitigating both direct and indirect injection attempts.

3.2 Jatmo Defense Method

Jatmo stands for “Jack of all trades, master of one.” [4] . This technique focuses on defending against prompt injection attacks on LLM-integrated applications. Jatmo deals with prompt injection attacks by creating task specific language models which are immune to prompt injection attacks. General purpose LLMs like GPT 3.5 are instruction tuned, which means they are trained to follow instructions in their input making them prone to following malicious prompts. Jatmo circumvents this process by fine-tuning a non-instruction-tuned base model to perform only one task. Therefore, the model ignores the injected malicious commands, embedded in user input, since it was never trained to recognize or follow arbitrary instructions. Jatmo method involves three main stages:

3.2.1 Dataset Collection

The first stage involves assembling a dataset of clean input examples which are relevant to the specific task that the model is expected to perform. This serves as the foundation for building a task-specific model that is safe from prompt injection. There are two approaches for this dataset collection:

Option A: Utilizing Existing Datasets For example, IMDB dataset for sentiment classification or the CNN/DM dataset for summarization.

Option B: Synthetic Dataset Generation When no suitable dataset is available, Jatmo uses a fully automated method to generate a synthetic dataset using teacher mode. This comprises of 3 steps:

1. Seed Generation: A small set of diverse initial examples (typically 10) are generated based on a natural language description of the task. These serve as the foundational seeds for further expansion.
2. Dataset Expansion: Each seed is used to guide the creation of additional examples. This iterative process ensures diversity while maintaining structural consistency across the dataset until a sufficiently large dataset is generated.
3. Input Normalization and Formatting: The generated examples are then standardized into a consistent format. This step ensures that inputs are uniformly structured, which is essential for effective fine-tuning and reliable performance of the task-specific model.

This dataset generation ensures flexibility and applicability across wide range of tasks and domains.

3.2.2 Output Generation

Once a dataset of input examples D_i has been collected or synthesized, the next step is to generate corresponding output labels R_i for fine-tuning the task-specific model. To do this, JATMO uses a reliable, instruction-tuned language model

which is called the teacher model (typically GPT-3.5-Turbo).

For each input Di , we pair it with a predefined task prompt PP and feed both into the teacher model. The model processes the combined input $P+Di$ and generates a high-quality response Ri tailored to the task. This can be represented as: $Ri=M(P+Di)$, where M is the teacher model.

This process produces a training dataset of input-output pairs (Di,Ri) , which are then used as supervised examples for fine-tuning. A key assumption here is that the teacher model only sees clean, benign inputs during this phase—ensuring it remains safe from prompt injection attacks.

By relying on a high-quality teacher model, JATMO guarantees that the generated dataset is accurate and consistent, providing a solid foundation for training robust task-specific models in the next stage.

3.2.3 Fine Tuning a Base Model

The last stage of the JATMO framework involves transforming a base language model into a task-specific system through constrained fine-tuning. This process results in a specialized model F that is resistant to adversarial prompt manipulation. This process involves two steps:

1. **Base Model Selection:** The procedure begins with a foundation model which lacks instruction-tuning (e.g., OpenAI’s davinci-002). This selection rule is critical, as models without pre-existing instruction-following capabilities provide a null starting condition for prompt injection vulnerabilities.

2. **Targeted Supervised Fine Tuning:** Using the curated dataset of input-output pairs (Di,Ri) , the model parameters are optimised to establish a direct mapping:

$$F(Di) \rightarrow Ri$$

Crucially, the training excludes any meta-instructions or task prompts, conditioning the model to interpret all input strictly as task-domain data.

The resultant model naturally ignores any instruction embedded in the input and treats all input strictly as data. Thus, it effectively prevents prompt injection attacks by design, while performing at the level similar to a general purpose instruction tuned model like GPT 3.5.

3.3 Evaluation of Defenses

Evaluating the effectiveness of defense methods is crucial to understand how well the LLMs perform under the pressure of malicious activity.

In a defense strategy, [15] introduced a defense-enhancing middle-ware architecture incorporating a GPT-4-based Attack Validator. This component was evaluated against real-world attack types, including prompt chaining, model probing, and bias injection, using a custom adversarial prompt corpus. Effectiveness was measured by the attack detection rate, where the GPT-4 wrapper achieved between 76% and 97% success, outperforming traditional filters like fuzzy matching or keyword scans.

The Moving Target Defense strategy, was evaluated based on its performance by using a version of Mistral LLM and creating a dataset covering eight classes of prompt injection attacks. The evaluation included quantitative metrics like accuracy, F1-score, and precision, alongside a direct measurement of attack success rates before and after defense. MTD dynamically altered model parameters and context during inference, which led to a significant drop in attack success from 78% to 15%, with only a modest increase in computational overhead and minimal impact on performance [14].

The Jatmo framework proposes a workaround solution by fine tuning non

instruction tuned LLMs for specific tasks, which naturally prevents them from misinterpreting injected prompts[4]. For its evaluation, the researchers evaluated Jatmo across seven NLP tasks, which included summarization, sentiment analysis and news summarization, using both original and synthetic datasets. Attack success rates were measured by inserting a series of adversarial instructions (e.g., directing the model to output “HACKED”) at different positions (start, middle, end) within the input. Remarkably, JATMO achieved comparable task performance to GPT-3.5-Turbo (within 2% quality difference), while reducing the prompt injection success rate from 87% to as low as 0.5%, with some tasks seeing zero successful attacks [4].

The HOUYI framework [5] provides a comprehensive approach to evaluating the robustness of Large Language Models (LLMs) against prompt injection attacks by simulating adversarial scenarios through an automated attack generation and optimization pipeline. Its evaluation methodology centers on the use of a genetic algorithm-based attack generation mechanism, which iteratively crafts adversarial prompts [16] aimed at breaking the model’s alignment and control boundaries. Through this process, HOUYI generates high-quality “chromosomes” (i.e., malicious prompts) that maximize the attack success rate while preserving linguistic fluency. The framework was tested in several open source LLMs, such as LLaMA, Alpaca, and Vicuna, and it was shown that even models aligned using reinforcement learning (such as Vicuna-13B) were vulnerable to injection attacks in more than 60% of the test cases. Notably, HOUYI was able to jail-break advanced instruction-tuned models, causing them to output toxic, harmful, or otherwise undesired responses despite their safety guardrails. The evaluation also included a comparative analysis of various LLM families, revealing discrepancies in robustness between base models, instruction-tuned models, and models fine-tuned with RLHF. Furthermore, HOUYI assessed black-box transferability

3.3 Evaluation of Defenses

by testing whether attack prompts generated for one model could successfully exploit another which highlighted the cross-model generalization of certain injection patterns. These evaluations underscore the importance of adversarial testing frameworks like HOUYI, which not only expose real-world vulnerabilities but also provide a benchmark for assessing the effectiveness of existing and future prompt injection defenses.

Chapter 4

Data

You might need a chapter (or, alternatively, a section in chapter “Methodology”) about your data, its characteristics and pre-processing. This chapter is particularly relevant if you are using a dataset that has not been previously described in detail.

But even if you do not use a dedicated “Data” chapter, you still need to describe all datasets you are using (e.g., in chapter “Methodology”).

Existing datasets need to be cited with the exact sources.

4.0.1 Amazon *All_{Beauty}Dataset*

For this study, I selected the publicly available Amazon Beauty Reviews dataset[17], which contains real-world customer feedback on a wide range of beauty and personal care products. This dataset includes detailed product reviews along with metadata such as review titles, star ratings, timestamps, and verified purchase status.

In view of the limit of our computational resources, I had to reduce the size of

the dataset. I extracted and grouped the reviews into sets of three. Each group simulates a natural use case in which a language model is expected to generate a concise summary paragraph from multiple customer reviews. Our final dataset consisted of 1500 review groups and summary samples.

The following steps were necessary to prepare the data for finetuning our model:

- **Grouping:** Reviews were grouped into logical sets of three, preserving their original language and order to maintain contextual coherence.
- **Cleaning:** Minimal cleaning was done to remove incomplete or corrupted entries, ensuring that the natural linguistic variation remained intact.
- **Formatting:** Each example was structured in OpenAI’s fine-tuning format, using the chat-completion schema. This included assigning roles (**system**, **user**, **assistant**) and ensuring that all prompts were purely task-specific without any natural language instructions like ”Summarize this.”
- **JATMO Style Compliance:** In line with the JATMO methodology, we avoided instruction-style prompting. The model was only exposed to direct user inputs in the form of grouped reviews, followed by assistant responses that reflected high-quality summaries.

This final dataset was saved in `.jsonl` format and contained approximately 1,500 review-summary pairs, totaling nearly 400,000 tokens.

Chapter 5

Methodology

To address the research question of how effective existing defense mechanisms are against prompt injection attacks in LLMs, this project adopts a comparative evaluation approach using adversarial testing. The methodology involves utilizing and extending the open-source HOUYI framework to generate optimized prompt injection attacks through genetic algorithms. These attacks will then be applied to various LLM configurations, with a primary focus on evaluating JATMO, a task-specific fine-tuned model designed to resist injected prompts. To ensure a fair evaluation, both clean and adversarial prompt datasets will be used, combining publicly available benchmarks (e.g., for summarization and sentiment analysis) with custom synthetic prompts. Performance metrics will include attack success rate, accuracy, and model stability under injection. The study will also examine cross-model transferability to assess how well attack prompts generalize across LLMs. Code implementation, model fine-tuning, and attack simulations will be executed in Python using frameworks such as Hugging Face Transformers and PyTorch. Limitations may include computational constraints and restricted access to proprietary instruction-tuned models, which will be addressed by focusing on open-source alternatives like LLaMA or Alpaca.

Chapter 6

Experiments

Give a complete technical description of your experiments, sufficient for another researcher to understand your experimental approach and reproduce your results.

(Note that experimental *results* and their discussion belong in the next chapter.)

Chapter 7

Results

Results first, using figures and tables, with little commentary and no interpretation.

Then detailed analysis and interpretation.

Link experimental results to your research questions or hypotheses, e.g., “The results in Table 3 clearly show that the answer to RQ2 is negative ...”.

Chapter 8

Conclusion

Here you must zoom back out to evaluate the thesis itself. Mention its contributions and main results (in brief form), its limitations and weaknesses, and possible future work.

References

- [1] Z. Wang, Z. Chu, T. V. Doan, S. Ni, M. Yang, and W. Zhang, “History, development, and principles of large language models: an introductory survey,” *AI and Ethics*, pp. 1–17, 2024. v, 3, 4
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017. v, 4, 5
- [3] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, K. Wang, and Y. Liu, “Jailbreaking chatgpt via prompt engineering: An empirical study,” *arXiv preprint arXiv:2305.13860*, 2023. v, 8
- [4] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, Z. Wei, E. Sun, B. Alomair, and D. Wagner, “Jatmo: Prompt injection defense by task-specific finetuning,” in *Computer Security – ESORICS 2024*, J. Garcia-Alfaro, R. Kozik, M. Choraś, and S. Katsikas, Eds. Cham: Springer Nature Switzerland, 2024, pp. 105–124. 1, 11, 15
- [5] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng *et al.*, “Prompt injection attack against llm-integrated applications,” *arXiv preprint arXiv:2306.05499*, 2023. 1, 15
- [6] I. L. A. et al., “Large language models (llm) and chatgpt: what will the

REFERENCES

- impact on nuclear medicine be?” *European Journal of Nuclear Medicine and Molecular Imaging*, vol. 50, p. 026113, 2023. 3
- [7] T. Mikolov, M. Karafiát, L. Burget, J. Cernocký, and S. Khudanpur, “Recurrent neural network based language model.” in *Interspeech*, vol. 2, no. 3. Makuhari, 2010, pp. 1045–1048. 3
- [8] L. Hu, Z. Liu, Z. Zhao, L. Hou, L. Nie, and J. Li, “A survey of knowledge enhanced pre-trained language models,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 4, pp. 1413–1430, 2023. 4
- [9] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020. 4
- [10] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023. 4
- [11] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (llm) security and privacy: The good, the bad, and the ugly,” *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S266729522400014X> 6, 7
- [12] X. Liu, N. Xu, M. Chen, and C. Xiao, “Autodan: Generating stealthy jailbreak prompts on aligned large language models,” *arXiv preprint arXiv:2310.04451*, 2023. 6

REFERENCES

- [13] A. Wan, E. Wallace, S. Shen, and D. Klein, “Poisoning language models during instruction tuning,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 35 413–35 425. 6
- [14] S. Panterino and M. Fellington, “Dynamic moving target defense for mitigating targeted llm prompt injection,” *Authorea Preprints*, 2024. 10, 14
- [15] O. Muliarevych, “Enhancing system security: Llm-driven defense against prompt injection vulnerabilities,” in *2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*. IEEE, 2024, pp. 420–423. 10, 14
- [16] G. Apruzzese, H. S. Anderson, S. Dambra, D. Freeman, F. Pierazzi, and K. Roundy, ““real attackers don’t compute gradients”: bridging the gap between adversarial ml research and practice,” in *2023 IEEE conference on secure and trustworthy machine learning (SaTML)*. IEEE, 2023, pp. 339–364. 15
- [17] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, “Bridging language and items for retrieval and recommendation,” *arXiv preprint arXiv:2403.03952*, 2024. 17